

Banking Case Study 2: EFT Payment Testing and Transaction Reconciliation

EFT Payment Testing and Account Transaction Reconciliation in a Digital Banking Platform

Case Study Summary

This case study demonstrates how I would test an EFT payment flow in a digital banking environment. The focus is on validating that money is correctly debited from the sender account, credited to the recipient account, recorded in transaction history, reflected in statements, stored correctly in the database and returned accurately through the API.

The purpose of this case study is to show how my regulated iGaming testing experience can transfer into banking and FinTech QA. In my current QA background, I validate financial logic, player balances, payouts and transactional workflows. Those same testing principles apply to banking systems where payment accuracy, reconciliation, transaction traceability and audit evidence are critical.

This case study uses a fictional banking platform and fictional data only.

1. Business Problem

In a banking system, an EFT payment must move money accurately from one account to another.

When a customer sends money, the system must:

Payment Activity	Expected System Behaviour
Debit sender account	Deduct the correct payment amount
Apply transaction fee	Deduct the correct bank fee where applicable
Credit recipient account	Add the correct amount to the recipient account
Create transaction records	Store the transaction for both sender and recipient
Update histories	Show the transaction in mobile banking and web banking
Update statements	Reflect the payment on the customer statement
Update admin records	Allow internal users to trace the transaction
Return API data	Return the correct payment status and values
Maintain audit trail	Store reference number, timestamp and status

If any part of this flow fails, the bank may show incorrect balances, incomplete statements or mismatched records between systems.

2. Why This Case Study Fits My Background

This case study fits my background because both iGaming and banking depend on accurate transactional records.

iGaming Testing Area	Banking Equivalent
Player balance before round	Sender balance before payment
Wager deduction	Payment debit
Payout credit	Recipient credit
Game round ID	Payment reference number
Game history	Customer transaction history
Platform history	Banking channel history
Back office record	Internal banking admin record
Database transaction record	Core banking transaction table
Certification evidence	Audit and compliance evidence

The domain changes, but the QA thinking remains similar. Both require balance accuracy, transaction traceability, record reconciliation, defect investigation, audit evidence, backend validation and financial risk awareness.

3. Business Risk

Risk Area	Possible Impact
Sender not debited	Customer sends money without funds leaving account
Recipient not credited	Recipient does not receive the payment
Duplicate debit	Sender is charged more than once
Incorrect fee	Bank charges are wrong
Incorrect balance	Customer sees an inaccurate account balance
Missing transaction history	Customer cannot trace the payment
Statement mismatch	Official statement differs from account history
API mismatch	Integrated systems receive incorrect transaction data
Database mismatch	Backend record does not match customer view
Audit trail failure	Payment cannot be investigated properly
Regulatory risk	Bank may fail audit or compliance checks

4. Test Objective

The objective of this case study is to validate that an EFT payment is processed accurately from sender to recipient.

The testing confirms that:

Validation Area	Objective
Sender opening balance	Confirm sender balance before payment
Recipient opening balance	Confirm recipient balance before payment
Payment amount	Confirm correct transfer amount
Transaction fee	Confirm correct fee deduction
Sender closing balance	Confirm sender balance after payment
Recipient closing balance	Confirm recipient balance after payment
Payment status	Confirm transaction status is completed
Payment reference	Confirm unique reference is created
Transaction history	Confirm sender and recipient histories are updated
Statement record	Confirm statement entries are correct
Admin portal	Confirm internal records match customer view
API response	Confirm payment data is returned correctly
Database record	Confirm backend transaction values are accurate

5. Test Scope

In Scope

Area	Description
Customer login	Confirm that the sender can access the banking platform
Sender balance validation	Confirm sender opening and closing balances
Recipient balance validation	Confirm recipient opening and closing balances
EFT payment creation	Confirm payment details can be entered
Payment confirmation	Confirm customer can approve payment
Fee validation	Confirm fee is applied correctly
Transaction reference	Confirm unique payment reference is created
Transaction history	Confirm payment appears in sender and recipient records
Statement validation	Confirm statement entries are correct
Admin portal validation	Confirm internal record is accurate
API validation	Confirm payment status and transaction details
SQL validation	Confirm database records match expected values

Reconciliation	Compare sender, recipient, statement, admin, API and database records
Defect reporting	Log any mismatch with evidence and risk

Out of Scope

Area	Reason
Real banking production systems	This is a fictional portfolio case study
Real customer accounts	No real customer data is used
Fraud rule engine testing	Fraud detection is not the focus of this case study
Security penetration testing	Security testing is outside the current scope
Interbank settlement testing	The case study focuses on customer facing EFT validation
Legal regulatory interpretation	The focus is QA validation, not legal advice

6. Test Environment

Item	Example Value
Application type	Demo digital banking platform
Platform	Web banking or mobile banking
Environment	QA environment
Sender account	Fictional funded account
Recipient account	Fictional active account
Currency	ZAR
Browser	Chrome, Edge or Firefox
API tool	Postman
Database	PostgreSQL or MySQL
Defect tracking	JIRA style defect tracking
Evidence	Screenshots, SQL results, API responses and reconciliation tables

7. Test Data

Field	Value
Sender customer ID	CUST1001
Sender account number	ACC1001
Recipient customer ID	CUST2001
Recipient account number	ACC2001
Sender opening balance	R10 000.00
Recipient opening balance	R2 500.00
Payment amount	R1 500.00
Transaction fee	R8.00
Expected sender closing balance	R8 492.00
Expected recipient closing balance	R4 000.00
Payment reference	EFT5001
Currency	ZAR
Transaction status	Completed

8. Payment Calculation Logic

Sender Balance Formula

Sender Closing Balance = Sender Opening Balance minus Payment Amount minus Transaction Fee

Example: R10 000.00 minus R1 500.00 minus R8.00 = R8 492.00

Recipient Balance Formula

Recipient Closing Balance = Recipient Opening Balance plus Payment Amount

Example: R2 500.00 plus R1 500.00 = R4 000.00

9. End to End EFT Payment Flow

Step	Banking Event	Expected Result
1	Sender logs into banking platform	Login is successful
2	Sender views account balance	Balance displays R10 000.00
3	Sender selects EFT payment	Payment screen opens
4	Sender enters recipient account	Recipient details are accepted
5	Sender enters R1 500.00 payment amount	Amount is accepted
6	System displays R8.00 transaction fee	Fee is shown before confirmation
7	Sender confirms payment	Payment is submitted
8	Sender account is debited	Balance becomes R8 492.00
9	Recipient account is credited	Balance becomes R4 000.00
10	Payment reference is generated	Reference EFT5001 is created
11	Sender transaction history updates	Debit transaction appears
12	Recipient transaction history updates	Credit transaction appears
13	Admin portal is checked	Internal payment record matches
14	API response is validated	API returns correct status and values
15	Database is checked	Backend records match expected values

10. Detailed Test Case 1

Test Case ID

TC BANK EFT 001

Test Case Title

Verify that an EFT payment debits the sender account correctly

Test Objective

To confirm that the sender account is debited by the payment amount and transaction fee after a successful EFT payment.

Preconditions

Precondition	Description
Sender account exists	A valid sender account is available
Sender account is funded	Sender balance is R10 000.00
Recipient account exists	Recipient account is active
EFT feature is enabled	Sender can create an EFT payment
Fee rule exists	R8.00 fee applies to the EFT payment
Database access is available	Tester can validate backend records

API access is available	Tester can validate payment response
-------------------------	--------------------------------------

Test Steps

Step	Action	Expected Result
1	Login using sender account CUST1001	Login is successful
2	Open account dashboard	Sender account is displayed
3	Confirm opening balance	Balance displays R10 000.00
4	Select EFT payment	Payment screen opens
5	Enter recipient account ACC2001	Recipient details are accepted
6	Enter payment amount of R1 500.00	Amount is accepted
7	Review payment confirmation screen	Payment amount and R8.00 fee are displayed
8	Confirm payment	Payment is processed successfully
9	Check sender account balance	Balance displays R8 492.00
10	Open sender transaction history	EFT debit and fee are displayed
11	Check admin portal	Sender debit record is available
12	Query database record	Sender debit values match expected result
13	Validate API response	API response returns completed status
14	Capture evidence	Screenshots, API response and SQL result are saved

Expected Result

The sender account should be debited by the payment amount and fee.

Field	Expected Value
Sender opening balance	R10 000.00
Payment amount	R1 500.00
Transaction fee	R8.00
Expected sender closing balance	R8 492.00

11. Detailed Test Case 2

Test Case ID

TC BANK EFT 002

Test Case Title

Verify that an EFT payment credits the recipient account correctly

Test Objective

To confirm that the recipient account is credited with the correct payment amount after a successful EFT payment.

Preconditions

Precondition	Description
Sender payment is completed	EFT payment has been submitted successfully
Recipient account exists	Recipient account ACC2001 is active
Recipient opening balance is known	Recipient balance is R2 500.00
Transaction history is available	Recipient history can be checked
Database access is available	Tester can validate backend records

Test Steps

Step	Action	Expected Result
1	Login or access recipient account record	Recipient account is available

2	Confirm recipient opening balance	Balance before payment is R2 500.00
3	Locate incoming EFT payment	Payment reference EFT5001 is visible
4	Confirm credited amount	Credit amount displays R1 500.00
5	Confirm recipient closing balance	Balance displays R4 000.00
6	Open recipient transaction history	Incoming EFT appears correctly
7	Check statement view	Credit transaction appears correctly
8	Check admin portal record	Recipient credit record is correct
9	Query database transaction record	Backend credit values match expected result
10	Validate API response	API confirms recipient credit
11	Capture evidence	Screenshots, SQL result and API response are saved

Expected Result

The recipient account should be credited with the payment amount.

Field	Expected Value
Recipient opening balance	R2 500.00
Credit amount	R1 500.00
Expected recipient closing balance	R4 000.00

12. Detailed Test Case 3

Test Case ID

TC BANK EFT 003

Test Case Title

Verify EFT payment reference and transaction status

Test Objective

To confirm that the EFT payment creates a unique transaction reference and that the payment status is correctly displayed as completed.

Test Steps

Step	Action	Expected Result
1	Complete an EFT payment	Payment is submitted successfully
2	Capture payment confirmation screen	Payment reference is displayed
3	Open sender transaction history	Same reference appears on sender record
4	Open recipient transaction history	Same reference appears on recipient record
5	Check admin portal	Same payment reference appears internally
6	Query database	Same reference exists in backend records
7	Validate API response	API returns the same payment reference
8	Confirm transaction status	Status is COMPLETED across all sources

Expected Result

The payment reference EFT5001 and status COMPLETED should appear consistently across all relevant systems.

13. Reconciliation Table

Field	Expected	Sender History	Recipient History	Statement	Admin Portal	API Response	Database	Result
Payment reference	EFT5001	EFT5001	EFT5001	EFT5001	EFT5001	EFT5001	EFT5001	Pass
Sender opening balance	R10 000.00	R10 000.00	N/A	R10 000.00	R10 000.00	R10 000.00	R10 000.00	Pass
Recipient opening balance	R2 500.00	N/A	R2 500.00	R2 500.00	R2 500.00	R2 500.00	R2 500.00	Pass
Payment amount	R1 500.00	R1 500.00	R1 500.00	R1 500.00	R1 500.00	R1 500.00	R1 500.00	Pass
Transaction fee	R8.00	R8.00	N/A	R8.00	R8.00	R8.00	R8.00	Pass
Sender closing balance	R8 492.00	R8 492.00	N/A	R8 492.00	R8 492.00	R8 492.00	R8 492.00	Pass
Recipient closing balance	R4 000.00	N/A	R4 000.00	R4 000.00	R4 000.00	R4 000.00	R4 000.00	Pass
Transaction status	Completed	Completed	Completed	Completed	Completed	Completed	Completed	Pass

14. SQL Validation Examples

Query 1: Validate sender debit transaction

```
SELECT
    transaction_reference,
    sender_account,
    recipient_account,
    transaction_type,
    amount,
    fee_amount,
    sender_balance_after,
    transaction_status,
    created_at
FROM bank_payment_transactions
WHERE transaction_reference = 'EFT5001'
AND sender_account = 'ACC1001';
```

Expected SQL Result

transaction_reference	sender_account	recipient_account	type	amount	fee_amount	sender_balance_after	status
EFT5001	ACC1001	ACC2001	DEBIT	1500.00	8.00	8492.00	COMPLETED

Query 2: Validate recipient credit transaction

```
SELECT
    transaction_reference,
    recipient_account,
    transaction_type,
    amount,
    recipient_balance_after,
    transaction_status,
    created_at
FROM bank_payment_transactions
WHERE transaction_reference = 'EFT5001'
AND recipient_account = 'ACC2001';
```

Expected SQL Result

transaction_reference	recipient_account	type	amount	recipient_balance_after	status
EFT5001	ACC2001	CREDIT	1500.00	4000.00	COMPLETED

Query 3: Validate no duplicate EFT payment was created

```
SELECT
    transaction_reference,
    COUNT(*) AS transaction_count
FROM bank_payment_transactions
WHERE transaction_reference = 'EFT5001'
GROUP BY transaction_reference;
```

Expected SQL Result

transaction_reference	transaction_count
EFT5001	2

The expected count is 2 because one debit record is created for the sender and one credit record is created for the recipient.

15. API Validation Examples

API Test 1: Submit EFT Payment

Request

POST /api/payments/eft

Request Body

```
{
  "senderAccount": "ACC1001",
  "recipientAccount": "ACC2001",
  "amount": 1500.00,
  "currency": "ZAR",
  "description": "Portfolio EFT test"
}
```

Expected Response

```
{
  "transactionReference": "EFT5001",
  "senderAccount": "ACC1001",
```



```

"recipientAccount": "ACC2001",
"amount": 1500.00,
"feeAmount": 8.00,
"currency": "ZAR",
"status": "COMPLETED"
}

```

API Assertions

Assertion	Expected Result
Status code	201 Created
transactionReference	EFT5001
senderAccount	ACC1001
recipientAccount	ACC2001
amount	1500.00
feeAmount	8.00
currency	ZAR
status	COMPLETED

API Test 2: Get EFT Payment Status

Request

```
GET /api/payments/eft/EFT5001
```

Expected Response

```

{
  "transactionReference": "EFT5001",
  "paymentType": "EFT",
  "senderAccount": "ACC1001",
  "recipientAccount": "ACC2001",
  "amount": 1500.00,
  "feeAmount": 8.00,
  "status": "COMPLETED",
  "createdAt": "2026-05-23T10:30:00"
}

```

API Assertions

Assertion	Expected Result
Status code	200 OK
paymentType	EFT
transactionReference	EFT5001
amount	1500.00
feeAmount	8.00
status	COMPLETED

16. Negative Test Scenarios

Test Case ID	Scenario	Expected Result
TC BANK EFT 004	Sender attempts EFT with insufficient balance	Payment is rejected
TC BANK EFT 005	Recipient account does not exist	Payment is rejected
TC BANK EFT 006	Duplicate payment request is submitted	Duplicate payment is prevented
TC BANK EFT 007	Fee is not deducted from sender	Defect is logged
TC BANK EFT 008	Sender is debited but recipient is not credited	Critical defect is logged
TC BANK EFT 009	Recipient is credited but sender is not	Critical defect is logged

	debited	
TC BANK EFT 010	Transaction status remains pending after completion	Defect is logged
TC BANK EFT 011	API returns completed but database shows failed	Defect is logged
TC BANK EFT 012	Statement does not show EFT payment	Defect is logged
TC BANK EFT 013	Admin portal shows incorrect recipient account	Defect is logged
TC BANK EFT 014	Payment reference is duplicated	Critical defect is logged

17. Sample Defect Report

Defect ID

DEF BANK EFT 001

Defect Title

Sender account debited but recipient account not credited after EFT payment

Severity

Critical

Priority

High

Environment

QA Banking Web Build 1.0.5

Module

EFT Payments

Related Test Case

TC BANK EFT 002

Description

After submitting an EFT payment of R1 500.00 from sender account ACC1001 to recipient account ACC2001, the sender account is debited successfully, but the recipient account is not credited.

The payment status displays as completed, but the recipient balance remains unchanged.

Steps to Reproduce

Step	Action
1	Login using sender account CUST1001
2	Confirm sender opening balance is R10 000.00
3	Create EFT payment to recipient account ACC2001
4	Enter payment amount of R1 500.00
5	Confirm the payment
6	Check sender account balance
7	Check recipient account balance
8	Check sender and recipient transaction histories
9	Query database records for transaction reference EFT5001
10	Compare API response with database records

Expected Result

The sender account should be debited and the recipient account should be credited.

Account	Expected Result
Sender account	R10 000.00 minus R1 500.00 minus R8.00 = R8 492.00
Recipient account	R2 500.00 plus R1 500.00 = R4 000.00

Actual Result

The sender account balance updates to R8 492.00. The recipient account balance remains R2 500.00. The payment status displays as Completed.

Evidence

Evidence	Description
Screenshot 1	Sender opening balance
Screenshot 2	EFT payment confirmation
Screenshot 3	Sender closing balance showing debit
Screenshot 4	Recipient balance unchanged
Screenshot 5	Payment status showing completed
SQL result	Sender debit record exists but recipient credit is missing
API response	Payment status returned as completed
Reconciliation table	Sender and recipient mismatch highlighted

Business Impact

This issue creates a serious financial discrepancy because funds leave the sender account but do not reach the recipient account. This may result in customer complaints, failed reconciliation, manual investigation and loss of trust in the payment system.

Compliance Risk

The system marks the payment as completed even though the full financial movement did not occur. This creates audit, reconciliation and financial integrity risk because the payment status does not reflect the true transaction state.

Recommendation

The EFT processing logic should be reviewed to confirm that debit and credit posting are completed as one controlled transaction. If recipient credit fails, the system should either reject the full payment or reverse the sender debit.

The defect should remain open until the following are retested successfully:

Retest Area	Expected Result
Sender debit	Correctly posted
Recipient credit	Correctly posted
Payment status	Only completed when both sides succeed
Database records	Debit and credit records both created
API response	Matches actual payment state
Transaction histories	Sender and recipient records updated

18. Test Execution Summary

Metric	Count
Total test cases	14
Passed	10
Failed	1
Blocked	0
Not run	3
Defects logged	1
Critical defects	1

High priority defects	1
-----------------------	---

19. Summary of Findings

Testing confirmed that the sender debit, transaction fee deduction and payment reference generation were working as expected. However, one critical issue was identified where the recipient account was not credited even though the sender account was debited and the payment status displayed as completed.

This defect creates a serious reconciliation issue because the payment record does not accurately represent the full movement of funds.

20. Final QA Recommendation

Based on the executed EFT payment test scope, the product should not proceed to release until the recipient credit defect has been resolved and retested.

The corrected build should confirm that an EFT payment only reaches Completed status when both the sender debit and recipient credit are successfully posted.

The retest should validate the payment across:

Source
Sender account balance
Recipient account balance
Sender transaction history
Recipient transaction history
Statement view
Admin portal
API response
Database record

21. Skills Demonstrated

Skill	How It Was Demonstrated
Functional testing	Tested EFT payment creation and confirmation
Financial validation	Verified sender debit, recipient credit and fee deduction
Reconciliation testing	Compared sender, recipient, statement, admin, API and database records
SQL validation	Queried backend transaction records
API testing	Validated EFT payment submission and payment status responses
Defect reporting	Logged a critical defect with clear reproduction steps
Risk analysis	Explained business and compliance impact
Backend validation	Confirmed whether debit and credit records were correctly created
Banking domain thinking	Applied payment flow logic to digital banking
Compliance awareness	Focused on auditability, traceability and transaction integrity

22. Portfolio Disclaimer

All examples in this case study are fictional and created for portfolio purposes. No real banking system, customer account, employer system, client system or confidential data is used. The case study is designed to demonstrate transferable QA skills from regulated transactional testing into banking and FinTech quality assurance.